

Rendering Stratejileri ve Native Entegrasyon

Ömer Faruk Yavuz

December 2025

Introduction

Modern frontend geliştirme pratiğinde, bir uygulamanın *nasıl* render edildiği ve *hangi platformlarda* çalıştığı, sadece performans ve SEO açısından değil, geliştiricinin günlük karar alma süreçleri açısından da kritik bir rol oynar. React ekosisteminde SSR, ISR, SPA, CSR ve SSG gibi farklı rendering stratejileri, aynı bileşen ağacını bambaşka teslimat ve yürütme modelleriyle son kullanıcıya ulaştırmayı mümkün kılar. Bu stratejilerin sınırlarını, güçlü ve zayıf yönlerini ve hangi problem tipinde hangi yaklaşımın tercih edilmesi gerektiğini anlamak, özellikle ölçeklenebilir web arayüzleri tasarlarlarken temel bir gerekliliktir.

Diğer yandan, React Native geliştiricileri, yalnızca JavaScript tarafında bileşen yazmaktan ibaret olmayan, Android ve iOS ekosistemleriyle iç içe geçmiş bir çalışma biçimine sahiptir. Uygulama yaşam döngüsü, layout ve yoğunluk (dp, dpi), asset yönetimi, AndroidManifest ve Info.plist yapılandırmaları, Gradle ve CocoaPods gibi araçlar ile React Native bridge mimarisi, bu bütünün vazgeçilmez parçalarıdır. Özellikle mevcut native sınıfların veya üçüncü parti kütüphanelerin JS tarafına bridge edilmesi, gerçek projelerde sıkça ihtiyaç duyulan bir entegrasyon senaryosudur.

Bu metnin amacı, bir yandan web tarafındaki farklı rendering stratejilerini somut senaryolar ve kısa teknik örnekler üzerinden özetlerken, diğer yandan da React Native geliştiricisinin Android ve iOS dünyasını “yeterince” tanımasını sağlayacak kavramsal bir çerçeve sunmaktır. Böylece okuyucu, hem *hangi rendering modelini ne zaman seçmesi gerektiği* konusunda daha bilinçli kararlar alabilecek, hem de React Native projelerinde karşılaştığı native odaklı sorunları okuyup yorumlayabilecek bir temel altyapıya sahip olacaktır.

SSR (Server-Side Rendering)

Kısa Tanım Her istek geldiğinde sayfa **sunucuda** HTML olarak üretilir, sonra tarayıcıya gönderilir.

Örnek Senaryo: E-ticaret Ürün Sayfası Bir e-ticaret siten var: `myshop.com`. Her ürünün stok durumu ve fiyatı sık sık değişiyor.

- Kullanıcı `/product/123` adresine girdiğinde:
 1. Sunucu veritabanından ürün 123'ün güncel fiyatını ve stok bilgisini çeker.
 2. React bileşenlerini server tarafında çalıştırır ve tamamen dolu bir HTML üretir.
 3. Tarayıcı bu HTML'i direkt gösterir, kullanıcı hemen ürün fotoğrafını, fiyatı, sepete ekle butonunu görür.
 4. Ardından tarayıcı tarafında JS yüklenir, sayfa *hydrate* olur ve interaktif hale gelir.
- Her kullanıcı için **gerçek zamanlı** stok/fiyat bilgisi göstermek istediğin için SSR mantıklıdır.

Teknik Örnek (Next.js ile)

```
export async function getServerSideProps(context) {  
  const { id } = context.params;  
  const product = await fetch('https://api.myshop.com/products/${id}')  
    .then(res => res.json());  
  
  return { props: { product } };  
}
```

Bu örnekte her istek geldiğinde `getServerSideProps` çalışır, ürünü çeker ve sayfa sunucuda render edilir.

SSR, HTML ve CSS İlişkisi

SSR Ne Yapar, Ne Yapmaz? Server-Side Rendering (SSR) sadece *HTML'in nereden ve ne zaman üretildiği* ile ilgilidir. Yani:

- CSR/SPA yaklaşımında HTML **tarayıcıda** (client-side) üretilir.
- SSR yaklaşımında HTML **sunucuda** (server-side) üretilir.

Ancak hangi yaklaşımı kullanırsanız kullanın:

- Tasarım dili (Tailwind, CSS Modules, styled-components, düz CSS vb.),
- Kullandığınız `className`'ler ve stil kuralları

aynı kalabilir. SSR, tasarımın “çirkin” ya da “güzel” olmasını belirlemez; sadece hazır HTML'in server'da mı, yoksa client'ta mı üretildiğini belirler.

Basit Bir SSR Örneği (Stilsiz) Aşağıdaki örnek sadece SSR mantığını göstermek için *bilerek* sade yazılmıştır; CSS eklenmemiştir:

```
export default function ProductPage({ product }) {
  return (
    <div>
      <h1>{product.name}</h1>
      <p>Price: {product.price}</p>
      <p>{product.inStock ? 'In stock' : 'Out of stock'}</p>
    </div>
  );
}
```

Bu haliyle üretilen HTML, tasarım açısından minimaldir; çünkü hiçbir `className` veya özel stil kullanılmamıştır. Ama bu, SSR yüzünden değil, örneğin basit tutulmuş olmasındandır.

Aynı Sayfanın “Gerçek Hayat” Versiyonu Gerçekte tam tersine, SSR kullanılan projelerde de zengin CSS, tasarım sistemi ve bileşen yapısı kullanılır. Örneğin:

```
export default function ProductPage({ product }) {
  return (
    <main className="page">
      <section className="product-card">
        <h1 className="product-title">{product.name}</h1>
        <p className="product-price">{product.price} </p>
        <p className={
          'product-stock ${product.inStock ? 'in-stock' : 'out-of-stock'}'
        }>
          {product.inStock ? 'Stokta var' : 'Stokta yok'}
        </p>
      </section>
    </main>
  );
}
```

Bu component:

- CSR/SPA ile render edilirse de,
- SSR ile server tarafında render edilip HTML olarak gönderilirse de

aynı JSX'ten, dolayısıyla aynı HTML ve aynı CSS kurallarından beslenir. Fark sadece *nerede* render edildiğidir.

CSS SSR’de Nasıl Çalışır? Normal bir projede global CSS kullanımında ne yapıyorsanız, SSR senaryosunda da aynısını yaparsınız. Örneğin Next.js’de:

- `styles/globals.css` gibi bir dosyada global stiller tanımlanır.
- `_app.tsx` veya `_app.js` içinde bu dosya import edilir.

```
// _app.tsx
import '../styles/globals.css';

export default function MyApp({ Component, pageProps }) {
  return <Component {...pageProps} />;
}
```

SSR sırasında Next.js:

- Sayfayı server tarafında HTML’e dönüştürür,
- `<head>` içinde ilgili `<link rel="stylesheet" ...>` veya stil bloklarını ekler,
- Tarayıcı bu HTML’i aldığı anda CSS dosyasını da yükleyip sayfayı *tasarım* şeklinde gösterir.

Dolayısıyla kullanıcı çıplak HTML görmek zorunda değildir; tasarım, CSR’de olduğu gibi burada da devrededir.

SSR ve CSR Farkını Özetlemek SSR ile CSR’nin farkını şöyle düşünebiliriz:

- **CSR / SPA:**
 - Tarayıcı ilk olarak çoğunlukla boş bir `<div id="root">` ve JS bundle alır.
 - React tarayıcıda çalışır, HTML client-side üretilir.
 - CSS daha önceden yüklenmişse, render işlemi sonrası görünüm şekillenir.
- **SSR:**
 - Sunucu, React bileşenlerinden HTML’i **server-side** üretir.
 - Tarayıcı doğrudan dolu bir HTML alır (içinde `className`’ler, semantik etiketler vb. ile).
 - Aynı CSS dosyalarıyla birlikte sayfa daha ilk anda tasarımlı görünür.
 - Ardından JS bundle yüklenir ve sayfa hydrate edilerek interaktif hale gelir.

Kısa Psikolojik Özet

- Kullanıcı açısından: SSR ve CSR doğru yapılandırıldığında *görünen tasarım* açısından aynı derecede güzel olabilir.
- SSR sadece şunu der: Bu tasarımlı sayfayı önce sunucuda üretip, sonra tarayıcıya göndereyim. Tasarımı bozmaz, sadece üretim yerini değiştirir.

ISR (Incremental Static Regeneration)

Kısa Tanım Sayfa ilk başta **statik** üretilir (SSG gibi), sonra tanımladığın süre doldukça arka planda **tekrar** oluşturulur. Kullanıcılar CDN hızında statik sayfa görür ama içerik belirli aralıklarla güncellenir.

Örnek Senaryo: Blog veya Haber Sitesi Bir teknoloji blogun var: `techblog.com`. Yazılar genelde saatlik/günlük aralıklarla güncelleniyor ama *anlık* değişmiyor.

- `/posts/react-18` URL'i için:
 1. Build anında tüm blog yazıları statik HTML'e dönüştürülür.
 2. Kullanıcılar bu sayfayı CDN'den **çok hızlı** alır.
 3. Sen panelden yazıyı güncellersin.
 4. `revalidate = 60` ise, sayfa en geç 60 saniye içinde arka planda yeniden oluşturulur.
 5. Eski kullanıcılar güncelleme bitene kadar eski versiyonu görür, yeni istekler güncel versiyonu almaya başlar.

Teknik Örnek (Next.js ile)

```
export async function getStaticProps() {
  const posts = await fetch('https://api.techblog.com/posts')
    .then(res => res.json());

  return {
    props: { posts },
    revalidate: 60, // 60 saniyede bir arka planda yenile
  };
}
```

Bu örnekte `posts` sayfası statik olarak üretilir, ancak en geç 60 saniyede bir arka planda güncellenir.

SPA (Single Page Application)

Kısa Tanım Uygulama temelde **tek bir HTML sayfasıdır**. Tüm navigasyon ve içerik değişimleri **tarayıcıda JavaScript** ile yönetilir; tam sayfa yenileme olmaz.

Örnek Senaryo: Admin Paneli / Dashboard Bir SaaS ürünün admin paneli var: `app.mycompany.com`.

- Kullanıcı giriş yapar, `/dashboard`, `/users`, `/settings` gibi sayfalar arasında gezer.
- Her geçişte tarayıcı *yeni bir HTML* istemez; sadece URL değişir, React Router yeni komponenti render eder.
- Grafikler, tablolar, filtreler yoğun interaktif; ama SEO önemli değil (kullanıcı login olmak zorunda).
- Bu tip uygulama için SPA mantıklıdır.

Teknik Örnek (Klasik React SPA)

```
// index.html içinde
<div id="root"></div>
```

```
// index.js
import { createRoot } from 'react-dom/client';
import App from './App';
```

```
createRoot(document.getElementById('root')).render(<App />);
```

Router ile farklı sayfalar gösterilir ama hepsi tek HTML içinde, tek bundle ile çalışır.

CSR (Client-Side Rendering)

Kısa Tanım Render işlemi **tamamen tarayıcıda** yapılır. Sunucu çoğunlukla sadece boş bir HTML iskeleti ve JS bundle gönderir. Saf SPA yaklaşımı genelde CSR olarak adlandırılır.

Örnek Senaryo: Public Ama SEO'su İkinci Planda Olan Uygulama
Örneğin `music-player.com` gibi bir web müzik çalar:

- Kullanıcı siteye girer, ilk anda sadece loading spinner görür.
- JS bundle yüklendikten sonra React uygulaması ayağa kalkar, API'den şarkı listesi çekilir.
- Tüm liste, player, playlist yönetimi **istemci tarafında** render edilir.
- Arama motorunun ne gördüğü kritik değil; önemli olan kullanıcı için zengin interaktif deneyimdir.

SSG (Static Site Generation)

Kısa Tanım Sayfalar **build anında bir kez** üretilir ve tamamen statik HTML/CSS/JS olarak servis edilir. İçerik, yeni bir deploy yapana kadar değişmez.

Örnek Senaryo: Dokümantasyon veya Katalog Sitesi

- docs.myframework.com gibi bir dokümantasyon sitesi düşün.
- Dokümanlar ayda yılda bir değişiyor.
- Build aşamasında bütün sayfalar HTML'e çevrilip CDN'e koyuluyor.
- Kullanıcılar inanılmaz hızlı sayfa yükleme deneyimi yaşıyor, SEO da çok iyi.
- İçerik değişince siteyi yeniden build edip deploy etmen yeterli.

Teknik Örnek (Next.js ile)

```
export async function getStaticProps() {
  const docs = await fetch('https://api.myframework.com/docs')
    .then(res => res.json());

  return { props: { docs } };
}
```

getStaticProps sadece build sırasında çalışır, runtime'da değil.

Hangi Durumda Hangisi Seçilir?

- **Hem SEO hem de her istekte güncel veri önemliyse:** Kullanıcıya özel veya çok sık değişen içerikler için **SSR** tercih edilir.
- **Hem SEO hem de çok hızlı sayfa yüklenmesi önemli,** fakat veriler *saniye bazında* değil, *dakika/saat bazında* güncelleniyorsa: **ISR** iyi bir dentedir (statik hız + periyodik güncelleme).
- **Yoğun etkileşimli, uygulama hissi veren arayüzlerde** (dashboard, admin panel, SaaS UI) ve **SEO kritik değilse:** **SPA/CSR** genellikle en uygun yaklaşımdır.
- **İçerik nadiren değişiyorsa** (dokümantasyon, kurumsal tanıtım sitesi vb.) ve **maksimum hız + basitlik** istiyorsan: **SSG** tercih edilir.

Android Temelleri

Bu bölüm, bir React Native geliştiricisinin Android tarafını “yeterince” anlaması için bilmesi gereken temel kavramları açıklar. Amaç, tam zamanlı Android geliştiricisi olmak değil, React Native projelerinde karşılaştığın native sorunları ve ayarları okuyup yönetebilecek seviyeye gelmektir.

Application, Activity, Fragment, View ve ViewGroup

Application

- Android uygulamasının en tepedeki global nesnesidir.
- React Native projelerinde genellikle `android/app/src/main/java/.../MainApplication.java` ya da `.kt` dosyasında tanımlıdır.
- React Native “host”u, bazı kütüphane başlatmaları (analytics, crash reporting vb.) çoğu zaman burada yapılır.

Activity

- Android tarafında “ekran” gibi düşünülebilir.
- React Native template’inde `MainActivity` varsayılan giriş noktasıdır.
- Çoğu React Native uygulaması tek bir Activity içinde yaşar; ekran geçişleri JS tarafında (örneğin `react-navigation`) yönetilir.
- Bazı kütüphaneler dökümantasyonlarında `MainActivity`’ye kod eklemeni ister (örn. splash screen, gesture handler’in eski sürümleri).

Fragment

- Activity içindeki alt ekran veya bölüm birimidir.
- Saf native Android uygulamalarında yoğun kullanılır, React Native projelerinde genellikle doğrudan temas etmezsin.
- Dokümanlarda ya da hata mesajlarında adını gördüğünde, Activity içindeki daha küçük görsel birim olarak düşünebilirsin.

View

- Android’de tüm görsel bileşenlerin temel sınıfıdır.
- Buton, metin alanı, resim gibi her şey bir `View` veya ondan türemiş bir sınıftır.
- React Native’deki `<View>` bileşeni, native tarafta bir Android `View` (çoğunlukla bir `ViewGroup`) olarak temsil edilir.

ViewGroup

- İçinde başka **View** nesneleri tutabilen konteyner sınıfıdır.
- Örnekler: **LinearLayout**, **FrameLayout**, **ConstraintLayout** vb.
- React Native'de kullandığın çoğu **<View>**, altına başka bileşenler alabildiği için davranış olarak **ViewGroup** gibidir.

React Native ile ilişki React Native tarafında bir kök view vardır (**ReactRootView** veya yeni mimaride Fabric root). Senin yazdığın tüm **<View>**, **<Text>**, **<ScrollView>** ağacı, bu kök view'un altında bir Android view ağacına dönüştürülür.

Layout ve Yoğunluk: dp, px ve DPI

dp (density-independent pixel)

- Android, çok farklı ekran çözünürlüğü ve yoğunluğuna (dpi) sahip cihazlarda tasarımın tutarlı görünmesi için **dp** (density-independent pixel) birimini kullanır.
- **dp**, doğrudan ekrandaki fiziksel piksel sayısını değil, “algılanan boyutu” temsil eden soyut bir birimdir. Böylece aynı **dp** değeri, farklı yoğunluklardaki cihazlarda yaklaşık olarak aynı fiziksel boyuta karşılık gelir.
- React Native’de style içinde verdiği sayısal değerler (örneğin `width: 100`, `padding: 16`) pratikte **dp** cinsinden değerlendirilir; bu sayede aynı tasarım telefondan telefona kaba hatlarıyla benzer görünür.

Neden önemli?

- Aynı tasarımın bazı cihazlarda “daha büyük”, bazılarında “daha küçük” ya da orantısız görünmesi, çoğu zaman ekran yoğunluğu (dpi) farklarından kaynaklanır; **dp** kavramını bilmek bu farkı doğru yorumlamayı sağlar.
- Kenar çizgilerinde tek tük piksel kaymaları, ince border’ların bazı ekranlarda tam 1 piksel gibi görünmemesi, çizim ve yuvarlama farklarıyla ilgilidir; **dp** ve **dpi** fikri kafanda net olursa bu küçük görsel sapmalar daha tahmin edilebilir ve kabul edilebilir hale gelir.

React Native’de İkon ve Görseller: DPI, Ölçekleme ve Asset Yönetimi

Bu bölümde React Native ile Android’de ikon ve görsel (image) kullanırken **dpi**, **ölçekleme** ve **asset yönetimi** açısından bilmen gereken temel noktalar özetlenmektedir.

Saf Android Dünyasında Klasik Yaklaşım

Native Android (saf Kotlin/Java) tarafında genellikle aynı ikonun farklı çözünürlükte kopyaları şu klasörlerde tutulur:

- mipmap-mdpi
- mipmap-hdpi
- mipmap-xhdpi
- mipmap-xxhdpi
- mipmap-xxxhdpi

Her klasörde, aynı ikonun farklı boyutlarda bir kopyası yer alır. Android, cihazın ekran yoğunluğuna (dpi) göre doğru klasörden uygun ikonu seçer.

Bu mekanizma özellikle:

- Uygulama ikonu (launcher icon),
- Bildirim ikonu (notification icon)

gibi **sistem seviyesindeki ikonlar** için hâlâ geçerli ve önemlidir.

React Native içindeki normal ekran ikonları/görseller için ise, bu klasörlerle doğrudan uğraşmazsın; React Native kendi asset sistemini kullanır.

React Native Image Asset Sistemi (Yerel PNG/JPG)

React Native’de yerel bir görsel şu şekilde kullanılabilir:

```
<Image
  source={require('../assets/icon.png')}
  style={{ width: 24, height: 24 }}
/>
```

Burada React Native, kendi asset çözümleme sistemini devreye sokar.

@2x ve @3x Mantığı

Aynı ikonun farklı çözünürlükleri şu şekilde hazırlanabilir:

- `icon.png` → 1x (örneğin 24x24)
- `icon@2x.png` → 2x (örneğin 48x48)
- `icon@3x.png` → 3x (örneğin 72x72)

Bu dosyalar aynı klasörde durur. React Native:

- Cihazın `PixelRatio` değerine (ekran yoğunluğuna) bakar,
- Otomatik olarak en uygun çözünürlüklü dosyayı seçer.

Önemli noktalar:

- Senin ekstra bir “mdpi/hdpi” klasörü oluşturman gerekmez.
- Tasarımda 24dp bir ikon kullanıyorsan:
 - `icon.png` 24x24,
 - `icon@2x.png` 48x48,
 - `icon@3x.png` 72x72

boyutlarında export etmek mantıklıdır.

- Eğer sadece tek, düşük çözünürlüklü bir PNG koyup onu büyük boyutta gösterirsen, yüksek yoğunluklu ekranlarda ikon bulanık görünebilir.

SVG Kullanımı (Vektörel İkonlar)

Vektörel ikonlar (özellikle SVG) kullanıldığında:

- İkon piksel tabanlı değil, vektör tabanlıdır.
- `<YourSvgIcon width={24} height={24} />` gibi kullanımlarda, farklı dpi değerine sahip cihazlarda da ikon net kalır.
- @2x, @3x gibi ek export'lara ihtiyaç duymazsın.

Bu yüzden:

- Tab bar ikonları,
- Header ikonları,
- Küçük illüstrasyonlar

gibi UI içi ikonlar için SVG veya vektör kütüphaneleri (`react-native-vector-icons` vb.) kullanmak **daha az uğraştırıcı ve daha esnek** bir çözümdür.

PNG/JPG ise genellikle:

- Fotoğraflar,
- Arka plan görselleri,
- Çok detaylı illüstrasyonlar

gibi vektörle ifade edilmesi zor içerikler için tercih edilir.

Özel Durum: Uygulama İkonu ve Notification Icon

React Native ekranlarında kullandığın normal ikonlar/görseller RN asset sistemi ile çözümlenirken, bazı **özel sistem ikonları** hâlâ Android'in klasik **res** klasör yapısına göre hazırlanmalıdır:

- **Launcher icon (uygulama ikonu):**
 - `mipmap-mdpi`, `mipmap-hdpi`, `mipmap-xhdpi`, `mipmap-xxhdpi`, `mipmap-xxxhdpi` klasörlerine farklı boyutlarda export edilir.
 - Genellikle Android Studio'nun "Asset Studio" aracı ile tek seferde bu set oluşturulabilir.
- **Notification icon:**
 - `drawable-*` klasörlerine konur.
 - Çoğu zaman tek renkli, şeffaf arka planlı ikonlar tercih edilir.

Bu ikonlar, React Native **Image** bileşeninden bağımsız, doğrudan Android'in kendi kaynak (resource) sistemi tarafından kullanılır.

AndroidManifest.xml ve İzinler

Genel yapı

- Dosya yolu: `android/app/src/main/AndroidManifest.xml`
- Bu dosya, uygulamanın:
 - Paket adını,
 - Hangi Activity'leri içerdiğini,
 - Hangi izinleri (permission) istediğini,
 - Hangi `intent`-leri karşılayabildiğini (örneğin deep link) tanımlar.

Intent Nedir?

Android’de **intent**, başka bir bileşenden (özellikle bir Activity’den) bir eylem talep etmek için kullanılan “niyet” veya “mesaj” objesidir. Bir intent, kabaca şunları taşır:

- **Ne yapılacak?** (aksiyon)
Örneğin bir ekran açmak, bir URL’yi görüntülemek, bir fotoğraf çekmek.
- **Hangi veriyle yapılacak?** (data)
Örneğin bir `https://` linki, `miray://app/...` gibi bir deep link, bir dosya yolu, bir metin vb.
- **Ek bilgiler** (extras)
İsteğe bağlı olarak, anahtar-değer çiftleri şeklinde ek parametreler.

Gerçek hayatta bir zarf hazırlamaya benzetilebilir:

- Zarfın üzerinde ne istediğin (aksiyon) ve hangi bilgilerle yapmak istediğin (data, extras) yazar.
- Bu zarfı ya doğrudan belirli bir kişiye gönderirsin (explicit intent), ya da “bu işi kim yapabiliyorsa o açsın” dersin (implicit intent).

Intent ve AndroidManifest.xml İlişkisi `AndroidManifest.xml` içindeki `<intent-filter>` etiketleri, bir Activity’nin hangi tür intent’leri kabul edebileceğini tanımlar. Örneğin bir deep link için:

```
<intent-filter>
  <action android:name="android.intent.action.VIEW" />
  <category android:name="android.intent.category.DEFAULT" />
  <category android:name="android.intent.category.BROWSABLE" />
  <data android:scheme="miray" android:host="app" />
</intent-filter>
```

Bu tanım şunu ifade eder:

- Aksiyon: `ACTION_VIEW`,
- Veri şeması: `miray://`,
- Host: `app`.

Yani biri `miray://app/...` şeklinde bir link için `ACTION_VIEW` intent’i oluşturduğunda, bu Activity o intent’i karşılayabilen bir aday hâline gelir (örneğin bir deep link tıklandığında uygulamanın ilgili ekranının açılması).

Temel etiketler

<manifest> Paket adı (**package**) ve genel kapsam burada tanımlanır.

<application> Uygulama simgesi, adı, teması ve iç Activity'ler bu etiketin içinde yer alır.

<activity> Her Activity için ayrı bir tanım yapılır. Ana ekran (launcher) Activity genellikle bir **intent-filter** ile işaretlenir:

```
<activity
  android:name=".MainActivity"
  android:exported="true">
  <intent-filter>
    <action android:name="android.intent.action.MAIN" />
    <category android:name="android.intent.category.LAUNCHER" />
  </intent-filter>
</activity>
```

İzinler (Permissions)

- Kamera, mikrofon, depolama erişimi gibi yetkiler **<uses-permission>** etiketleriyle istenir:

```
<uses-permission android:name="android.permission.CAMERA" />
<uses-permission android:name="android.permission.RECORD_AUDIO" />
```

- React Native'de JS tarafında izin istemeden önce, ilgili iznin manifest'te tanımlı olması gerekir; aksi hâlde izin alınamaz veya uygulama beklenmedik davranış gösterebilir.

Deep Link Nedir?

Deep link, bir uygulamayı yalnızca açmak yerine, uygulamanın içindeki belirli bir ekrana veya kaynağa doğrudan gitmek için kullanılan linktir. Normal bir link genellikle ana sayfayı açarken, deep link belirli bir “iç sayfa” hedefler.

- Örnek (web link): `https://instagram.com`
Tarayıcıda Instagram ana sayfasını açar.
- Örnek (deep link): `instagram://user?username=omer` veya `https://instagram.com/omer`
Uygulama yüklüyse doğrudan ilgili kullanıcının profil ekranını açar.

React Native / Android bağlamında:

- Deep link, genellikle özel bir şema ve yol ile tanımlanır:
 - `miray://app/home` → Ana ekran
 - `miray://app/date/123` → ID’si 123 olan date detay ekranı
- Kullanıcı bu linke tıkladığında Android bir `Intent` oluşturur (örneğin `ACTION_VIEW` + `miray://app/date/123` URI’si ile) ve `AndroidManifest.xml` içindeki uygun `<intent-filter>` tanımına göre ilgili Activity’yi başlatır.
- Activity, gelen URI’yi okuyarak (örneğin `/date/123` yolunu) React Native tarafında doğru ekrana navigate edebilir.

Intent ve Deep Link'in Bağlantısı: Akış Örneği

Deep link ile Android **Intent** mekanizmasının birlikte nasıl çalıştığını aşağıdaki adım adım akış üzerinden görebiliriz:

1. **Kullanıcı bir deep link'e tıklar:**

- Örneğin: `miray://app/date/123`

2. **Android, bu linki yorumlar:**

- Bu, bir şeyi görüntüleme isteğidir (`ACTION_VIEW`).
- Veri (data) kısmı: `miray://app/date/123` URI'sidir.

3. **Sistem, bu bilgilerle bir Intent oluşturur:**

- **Action:** `Intent.ACTION_VIEW`
- **Data:** `miray://app/date/123`

4. **Android, `AndroidManifest.xml` içindeki `<intent-filter>` tanımlarına bakar:**

- “Bu tür bir Intent'i (aksiyon = `ACTION_VIEW` ve veri şeması = `miray://app/...`) kim karşılayabiliyor?”
- Uygun `intent-filter` ile eşleşen Activity adayları bulunur.

5. **Uygun Activity, bu Intent ile başlatılır:**

- Örneğin: `DeepLinkActivity` veya ana `MainActivity`.
- Activity, kendisine iletilen Intent'i `getIntent()` ile alır.

6. **Activity içinden URI okunur ve React Native tarafına yönlendirilir:**

- Activity, Intent içindeki URI'yi okur: `/date/123` gibi path bilgisi çıkarılır.
- Bu bilgi, React Native tarafına (örneğin React Navigation'a) aktarılır.
- Navigation katmanı, `DateDetailScreen` gibi ilgili ekrana, `id = 123` parametresiyle yönlendirme yapar.

Gradle ve Build Sistemi

Önemli dosyalar

- Proje seviyesi: `android/build.gradle`
- Uygulama seviyesi: `android/app/build.gradle`

Proje Seviyesi ve Uygulama Seviyesi `build.gradle` Farkı

Gradle yapılandırması Android tarafında iki ana katmanda ele alınır:

Proje seviyesi `build.gradle` (`android/build.gradle`)

- Tüm Android projesi için geçerli **genel ayarları** içerir.
- Örneğin:
 - Hangi Gradle eklentilerinin (`com.android.application`, `com.android.library`, vb.) kullanılacağı,
 - Hangi repository’lerden (`mavenCentral()`, `google()` vb.) bağımlılık indirileceği,
 - Bazı ortak/config değerlerinin üst düzeyde tanımlanması.
- Bir projede birden fazla modül (örneğin `app`, `library` vb.) varsa, bu dosya onların hepsini kapsayan “top-level” yapılandırmadır.

Uygulama seviyesi `build.gradle` (`android/app/build.gradle`)

- Sadece `app` modülüne ait **uygulama özelinde** ayarları içerir.
- React Native projesinde asıl Android uygulama modülü burasıdır.
- Örneğin:
 - `applicationId` (uygulamanın paket/kimlik adı),
 - `minSdkVersion`, `targetSdkVersion`,
 - `versionCode` ve `versionName`,
 - `buildTypes` (debug/release için ayrı ayarlar),
 - Uygulamanın kullandığı kütüphaneler (**dependencies** bloğu).
- Kısaca: Bu dosya, `app` modülünün Android uygulaması olarak nasıl derleneceğini ve hangi bağımlılıklarla çalışacağını tanımlar.

Özetle; proje seviyesi `build.gradle`, tüm modülleri kapsayan üst seviye Gradle ayarlarını; uygulama seviyesi `android/app/build.gradle` ise doğrudan React Native uygulama modülünün (Android `app`’in) ayrıntılı derleme ve bağımlılık yapılandırmasını içerir.

Temel komutlar

React Native geliştiricisi olarak pratikte en çok işine yarayacak **adb** komutları aşağıdaki gibidir:

adb devices Bağlı fiziksel cihaz ve emülatörleri listeler. Birden fazla cihaz bağlhyrsa, her biri için bir “serial” değeri gösterilir.

adb logcat Cihaz loglarını görüntüler. React Native ve native hataları buradan takip edilebilir. Belirli tag'lere göre filtrelemek de mümkündür. Örneğin:

```
adb logcat *:S ReactNativeJS:V
```

sadece **ReactNativeJS** loglarını ayrıntılı (verbose) olarak gösterir.

adb reverse tcp:8081 tcp:8081 Gerçek cihaz kullanırken, cihazın 8081 portunu geliştirme makinesindeki Metro bundler'a yönlendirir. React Native geliştirme sırasında “JS bundle'a bağlanamama” sorunlarında sıkça kullanılır.

adb reverse --remove-all Tanımlanmış tüm **reverse** yönlendirmelerini temizler. Ağ sorunlarını giderirken faydalıdır.

adb shell Cihazın içinde bir kabuk (shell) açar. Bu shell içinde ek komutlar (pm, ls, ps vb.) çalıştırılabilir.

adb shell pm clear <paket_adı> Belirtilen paket için uygulamanın verisini (cache, storage, shared preferences vb.) sıfırlar. Örneğin:

```
adb shell pm clear com.miray
```

ile **com.miray** paketinin tüm verileri temizlenir.

adb install <apk_dosyası> Verilen APK dosyasını cihaza yükler. Örneğin:

```
adb install app-debug.apk
```

adb uninstall <paket_adı> Belirtilen paket adındaki uygulamayı cihazdan kaldırır. Örneğin:

```
adb uninstall com.miray
```

adb kill-server / adb start-server ADB sunucusunu sırasıyla durdurur ve yeniden başlatır. Cihazın görünmemesi veya bağlantı sorunlarında “reset” etkisi yaratır.

adb -s <serial> <komut> Birden fazla cihaz/emülatör bağlhyken, belirli bir cihaza komut göndermek için kullanılır. Örneğin:

```
adb -s emulator-5554 logcat
```

sadece ilgili emülatörün loglarını gösterir.

iOS Temelleri (React Native Geliştiricisi İçin)

Bu bölüm, bir React Native geliştiricisinin iOS tarafında kavramsal olarak hakim olması gereken başlıkları içerir.

UIViewController, UIView ve Safe Area

UIViewController

- iOS tarafında ekran mantığını temsil eder.
- Her ekran tipik olarak bir `UIViewController` ile ilişkilidir.
- React Native uygulamasının kökünde de, uygulamayı taşıyan bir `UIViewController` bulunur.

UIView

- Tüm görsel bileşenlerin temel sınıfıdır.
- React Native'deki `<View>` bileşenleri, native tarafta birer `UIView` olarak oluşturulur.

Safe Area

- iPhone X ve sonrası cihazlardaki çentik, alt “home indicator” barı gibi alanları hesaba katan, “güvenli çizim alanı”dır.
- React Native'deki `SafeAreaView` bileşeni, bu güvenli alanı dikkate alarak içerik yerleşimi yapar.
- Üstten veya alttan kesilmiş gibi görünen içerik sorunlarını anlamak için Safe Area kavramı önemlidir.

Uygulama Yaşam Döngüsü (App Lifecycle)

Durumlar

- **Active:** Uygulama ekranda ve kullanıcı ile etkileşim hâlinde.
- **Inactive:** Geçiş durumlarında (örneğin bir sistem popup'ı ekrana geldiğinde) görülen ara durum.
- **Background:** Uygulama arka plana alınmış ancak hâlâ bellekte.
- **Terminated:** Uygulama tamamen kapatılmış, tekrar açıldığında baştan başlar.

Info.plist, İzinler ve URL Schemes

Info.plist nedir?

- Dosya yolu: `ios/YourApp/Info.plist`
- Uygulamanın yapılandırma bilgilerini içerir:
 - İzin açıklamaları (kamera, mikrofon, konum vb.),
 - URL şemaları (deep link),
 - Uygulama adı, ikon, sürüm bilgisi gibi metadata.

İzin açıklamaları (Usage Descriptions)

- Örneğin kamera ve mikrofon izinleri için:

```
<key>NSCameraUsageDescription</key>
<string>Kamera ile fotoğraf çekmek için gerekli.</string>

<key>NSMicrophoneUsageDescription</key>
<string>Ses kaydı için mikrofon erişimi gerekiyor.</string>
```

- Bu açıklamalar olmadan kamera veya mikrofon gibi hassas kaynaklara erişmeye çalışmak, hata veya reddedilmiş izinlerle sonuçlanabilir.

URL Schemes (Derin Bağlantılar)

- Uygulamanın özel URL şemalarını tanımlamak için kullanılır:

```
<key>CFBundleURLTypes</key>
<array>
  <dict>
    <key>CFBundleURLSchemes</key>
    <array>
      <string>miray</string>
    </array>
  </dict>
</array>
```



```
</dict>  
</array>
```

- Bu ayar sayesinde `miray://...` şeklindeki linkler uygulamayı açabilir.

CocoaPods ve Podfile

CocoaPods rolü

- iOS tarafında bağımlılık yönetimi için kullanılan paket yöneticisidir.
- React Native projelerinde, native kütüphaneler (Firebase, Maps, vb.) genellikle CocoaPods üzerinden eklenir.

Önemli dosya ve klasörler

- `ios/Podfile`: Pod tanımlarının bulunduğu dosya.
- `ios/Pods/`: Pod kurulumu sonrası oluşan bağımlılık klasörü.
- `YourApp.xcworkspace`: Xcode'da açılması gereken workspace dosyası (Pod'lar eklendikten sonra `.xcodproj` yerine bu kullanılmalıdır).

Temel komutlar

CocoaPods ile çalışırken pratikte en çok kullanılan komutlar şunlardır:

`pod install` Geçerli dizindeki `Podfile` dosyasını okuyup, tanımlı Pod bağımlılıklarını yükler. İlk kez Pod kurulumunda veya `Podfile` değiştiğinde kullanılır. React Native projelerinde genellikle:

```
cd ios
pod install
```

`pod update` Mevcut Pod'ların sürümlerini, `Podfile` ve repository'lerdeki güncel sürümlere göre günceller. Daha agresif bir işlemdir; tüm Pod'ları güncellemek yerine, çoğu zaman belirli bir Pod ismiyle kullanmak daha güvenlidir:

```
pod update
pod update Firebase/Auth
```

`pod repo update` CocoaPods'un yerel spec repository'sini günceller. Yani "Hangi Pod'un hangi sürümü var?" bilgisini tazeler. Bazı durumlarda `pod install` öncesi şu şekilde kullanılır:

```
pod repo update
pod install
```

`pod deintegrate` Xcode projesi ile Pod'lar arasındaki entegrasyonu kaldırır; Pod referanslarını temizler. Çok karışmış veya bozulmuş Pod yapılandırmalarını sıfırlamak için kullanılır. Genellikle ardından `pod install` ile temiz bir kurulum yapılır:

```
pod deintegrate
pod install
```

`pod cache clean --all` CocoaPods önbelleğini temizler. Pod cache'inde bozuk veya eski paketler kaldığında, sıfırdan indirmek için kullanılır:

```
pod cache clean --all
```

`pod outdated` Kurulu Pod'lar ile mevcut kullanılabilir sürümler arasındaki farkları gösterir. Hangi Pod'ların güncel olmadığını görmek için kullanışlıdır:

```
pod outdated
```

`pod search <isim>` Belirli bir Pod'u isimle aramak için kullanılır. Örneğin:

```
pod search Firebase
```

ilgili Pod'lar ve açıklamalarını listeler.

`pod env` CocoaPods ortamı hakkında bilgi verir (versiyon, kullanılan dizinler, repo konumları vb.). Sorun giderme sırasında ortam bilgisi paylaşmak gerektiğinde faydalıdır.

`pod --version` Yüklü CocoaPods sürümünü gösterir:

```
pod --version
```

React Native ile ilişkisi

- Birçok React Native kütüphanesi, iOS tarafında Pod eklenmesini gerektirir.
- Dokümanlarda “`cd ios && pod install`” uyarısı görmek bu yüzdendir.
- Pod ve Xcode tarafındaki hataları okuyabilmek, iOS build sorunlarını çözmeyi kolaylaştırır.

React Native ve Native Köprü (Bridge) Mantığı

JS Dünyası ve Native Dünyası

İki temel taraf

- **JS tarafı:**
 - React bileşenleri, iş mantığı, durum yönetimi (state, Redux, vb.) burada bulunur.
- **Native tarafı:**
 - Android ve iOS view'ları,
 - Platform API'lerine erişim (kamera, sensörler, bildirimler, dosya sistemi, vb.).

Bridge, Native Modules ve View Managers

Bridge mantığı (genel)

- React Native, JS kodu ile native kod arasında bir köprü (bridge) katmanı kullanır.
- JS tarafı bir fonksiyon çağırdığında, bu çağrı köprü üzerinden native tarafına iletilir; native taraf bir olay tetiklediğinde, köprü üzerinden JS tarafına event gönderilebilir.

Native Modules

- JS tarafından çağrılabilen, ama asıl implementasyonu native (Kotlin/Java veya Swift/Objective-C) olan fonksiyon setleridir.
- Örnekler:
 - Cihaz bilgisi kütüphaneleri (örneğin `react-native-device-info`),
 - Dosya sistemi erişimi (`react-native-fs`),
 - Özel iş mantığı taşıyan native servisler.
- React Native kütüphanelerinin çoğu, hem JS hem de native tarafında kod içerir.
- Dökümantasyonlarda:
 - `MainApplication` veya `AppDelegate` içine ekleme yapman istenebilir,
 - `AndroidManifest.xml`, `Info.plist`, `Podfile` veya `build.gradle` üzerinde değişiklik yapman gerekebilir.

Bridge Örneği: Basit Native Module (Android ve iOS)

Bu bölümde, React Native ile hem Android hem de iOS tarafında aynı JavaScript API'si üzerinden çalışan basit bir **Native Module** (`MyMathModule`) örneği gösterilmektedir.

Amaç: JS tarafında

```
MyMathModule.add(3, 4) -> Promise<number>
```

şeklinde kullanıldığında, hem Android (Kotlin) hem iOS (Swift) tarafında native kodun çalışmasıdır.

JavaScript Tarafı (Ortak Wrapper)

Önce Native Module için JS/TS wrapper yazılır. Bu dosya her iki platform için ortaktır.

```
// src/native/MyMathModule.ts
import { NativeModules } from 'react-native';

type MyMathModuleType = {
  add(a: number, b: number): Promise<number>;
};

const { MyMathModule } = NativeModules;

// Type assertion to get typed module in TS
export default MyMathModule as MyMathModuleType;
```

Kullanım örneği:

```
import MyMathModule from './native/MyMathModule';

async function demo() {
  const result = await MyMathModule.add(3, 4);
  console.log('Native result:', result); // 7
}
```

Android Bridge Örneği (Kotlin)

Native Module Sınıfı

Örnek dosya yolu: android/app/src/main/java/com/yourapp/MyMathModule.kt

```
package com.yourapp

import com.facebook.react.bridge.ReactApplicationContext
import com.facebook.react.bridge.ReactContextBaseJavaModule
import com.facebook.react.bridge.ReactMethod
import com.facebook.react.bridge.Promise

class MyMathModule(reactContext: ReactApplicationContext) :
    ReactContextBaseJavaModule(reactContext) {

    // Name visible on JS side: NativeModules.MyMathModule
    override fun getName(): String = "MyMathModule"

    // Promise-based method exposed to JS
    @ReactMethod
    fun add(a: Int, b: Int, promise: Promise) {
        val result = a + b
        promise.resolve(result)
    }
}
```

Package Sınıfı

Aynı paket altına bir ReactPackage implementasyonu eklenir:

```
// android/app/src/main/java/com/yourapp/MyMathPackage.kt
package com.yourapp

import com.facebook.react.ReactPackage
import com.facebook.react.bridge.NativeModule
import com.facebook.react.bridge.ReactApplicationContext
import com.facebook.react.uimanager.ViewManager

class MyMathPackage : ReactPackage {

    override fun createNativeModules(
        reactContext: ReactApplicationContext
    ): List<NativeModule> {
        return listOf(
            MyMathModule(reactContext)
        )
    }
}
```

```

    }

    override fun createViewManagers(
        reactContext: ReactApplicationContext
    ): List<ViewManager<*, *>> {
        return emptyList() // No native views in this example
    }
}

```

MainApplication İçine Paketin Eklenmesi

android/app/src/main/java/.../MainApplication.java (veya .kt) içinde getPackages() kısmına eklenir.

Java örneği:

```

import com.yourapp.MyMathPackage;

@Override
protected List<ReactPackage> getPackages() {
    List<ReactPackage> packages = new PackageList(this).getPackages();
    // Manually added packages (non-autolinked) go here:
    packages.add(new MyMathPackage());
    return packages;
}

```

Bu adımla, Android tarafında MyMathModule JS tarafından erişilebilir hale gelir.

iOS Bridge Örneği (Swift)

Bridging Header Kontrolü

Swift kullanılıyorsa, bir **bridging header** dosyası olmalıdır (örneğin `YourProject-Bridging-Header.h`):

```
// YourProject-Bridging-Header.h
#import <React/RCTBridgeModule.h>
```

Swift Module Sınıfı

Örnek dosya: `ios/MyMathModule.swift`

```
import Foundation

@objc(MyMathModule)
class MyMathModule: NSObject {

    // Whether the module needs to be initialized on the main thread
    @objc
    static func requiresMainQueueSetup() -> Bool {
        return false
    }

    // add(a, b) -> Promise<number> for JS
    @objc(add:withB:withResolver:withRejecter:)
    func add(
        a: NSNumber,
        b: NSNumber,
        resolve: RCTPromiseResolveBlock,
        reject: RCTPromiseRejectBlock
    ) {
        let result = a.intValue + b.intValue
        resolve(result)
    }
}
```

Buradaki `@objc(MyMathModule)` ismi, JS tarafında `NativeModules.MyMathModule` olarak erişilen isimle eşleşir.

Obj-C Module Deklarasyonu (Swift + RN için Yaygın Pattern)

Bazı kurulumlarda Swift modülünü Obj-C üzerinden deklare etmek için bir `.m` dosyası kullanılır. Örneğin `MyMathModule.m`:

```
#import <React/RCTBridgeModule.h>

@interface RCT_EXTERN_MODULE(MyMathModule, NSObject)
```

```
RCT_EXTERN_METHOD(  
  add:(nonnull NSNumber *)a  
    withB:(nonnull NSNumber *)b  
    withResolver:(RCTPromiseResolveBlock)resolve  
    withRejecter:(RCTPromiseRejectBlock)reject  
)  
  
@end
```

Bu deklarasyon, Swift ile yazılmış sınıfın React Native bridge tarafından doğru şekilde görülmesini sağlar.

JavaScript Tarafında Modülü Kullanmak

Hem Android hem iOS tarafında modül tanımlandıktan sonra, JS/TS tarafındaki kullanım aynıdır:

```
// Anywhere in JS/TS
import MyMathModule from './native/MyMathModule';

async function testNativeAdd() {
  try {
    const result = await MyMathModule.add(10, 32);
    console.log('Result from native:', result); // 42
  } catch (e) {
    console.error('Native error:', e);
  }
}
```

Bu örnek, aşağıdaki köprülemeyi gerçekleştirmiş olur:

- JS/TS tarafında `MyMathModule.add(a, b)` çağrısı yapılır.
- React Native bridge, çağrıyı platforma göre:
 - Android'de Kotlin ile yazılmış `MyMathModule.add`,
 - iOS'ta Swift ile yazılmış `MyMathModule.add`implementasyonlarına yönlendirir.
- Her iki platform da sonucu Promise üzerinden JS'e geri döndürür.

Böylece tek bir JavaScript API'si ile hem Android hem de iOS tarafında native kod çalıştıran minimal bir bridge örneği elde edilmiş olur.

Var Olan Native Sınıfı / Kütüphaneyi Bridge'lemek

Amaç: GitHub gibi bir yerden bulduğun bir Java/Kotlin/Swift kodunu React Native içinden sanki normal JS fonksiyonuymuş gibi kullanmak.

Tek Kotlin Sınıfını Bridge'lemek (Basit Örnek)

Elimizdeki Kotlin sınıfı

```
// com/example/Slugifier.kt
package com.example

object Slugifier {
    fun toSlug(input: String): String {
        return input
            .trim()
            .lowercase()
            .replace(Regex("\\s+"), "-")
            .replace(Regex("[^a-z0-9-]"), "")
    }
}
```

Bunu JS tarafında şöyle kullanmak istiyoruz:

```
const result = await SlugModule.toSlug("Merhaba Dünya!");
// "merhaba-dunya" dönsün
```

Sınıfı Android projesine dahil etmek

- Seçenek A – Kaynak kodu kopyalamak:

- Slugifier.kt dosyasını android/app/src/main/java/com/example/Slugifier.kt altına kopyala.
- Paket adı package com.example olarak kalır.

- Seçenek B – Gradle dependency olarak eklemek:

```
// android/app/build.gradle
dependencies {
    implementation "com.example:slug-lib:1.0.0"
}
```

Bu durumda sınıf kütüphane içinden gelir, sen sadece import edersin.

Bridge (Native Module) sınıfı yazmak

```
// android/app/src/main/java/com/yourapp/SlugModule.kt
package com.yourapp

import com.facebook.react.bridge.*
import com.example.Slugifier // Var olan Kotlin sınıfı

class SlugModule(reactContext: ReactApplicationContext) :
    ReactContextBaseJavaModule(reactContext) {

    override fun getName(): String = "SlugModule"

    @ReactMethod
    fun toSlug(text: String, promise: Promise) {
        try {
            val result = Slugifier.toSlug(text)
            promise.resolve(result)
        } catch (e: Exception) {
            promise.reject("SLUG_ERROR", e)
        }
    }
}
```

- JS `SlugModule.toSlug(...)` çağırır.
- Native Module içinden `Slugifier.toSlug(...)` çağırır.
- Sonuç `promise.resolve(...)` ile JS'e geri döner.

Paketi tanımlamak ve MainApplication içine eklemek

```
// android/app/src/main/java/com/yourapp/SlugPackage.kt
package com.yourapp

import com.facebook.react.ReactPackage
import com.facebook.react.bridge.NativeModule
import com.facebook.react.bridge.ReactApplicationContext
import com.facebook.react.uimanager.ViewManager

class SlugPackage : ReactPackage {
    override fun createNativeModules(
        reactContext: ReactApplicationContext
    ): List<NativeModule> {
        return listOf(SlugModule(reactContext))
    }

    override fun createViewManagers(
        reactContext: ReactApplicationContext
    ): List<ViewManager<*, *>> {
        return emptyList()
    }
}

// MainApplication.java içinde
import com.yourapp.SlugPackage;

@Override
protected List<ReactPackage> getPackages() {
    List<ReactPackage> packages = new PackageList(this).getPackages();
    packages.add(new SlugPackage());
    return packages;
}
```

JS tarafında modülü kullanmak

```
// src/native/SlugModule.ts
import { NativeModules } from 'react-native';

type SlugModuleType = {
  toSlug(text: string): Promise<string>;
};

const { SlugModule } = NativeModules;

export default SlugModule as SlugModuleType;

import SlugModule from './native/SlugModule';

async function example() {
  const s = await SlugModule.toSlug('Merhaba Dünya!');
  console.log(s); // "merhaba-dunya"
}
```

Özet: Mevcut Kotlin sınıfına dokunmadan, sadece üstüne bir **bridge** (Native Module) yazarak JS'ten erişilebilir hâle getirdik.

Birden Fazla .kt Dosyasından Oluşan Kütüphane

Gerçek hayatta kütüphanen genelde tek dosya değildir:

```
kotlin-lib/  
  src/main/kotlin/com/example/smartai/  
    SmartEngine.kt  
    SmartConfig.kt  
    SmartLogger.kt  
  internals/  
    Tokenizer.kt  
    Normalizer.kt  
  util/  
    StringExt.kt
```

Bridge açısından fark yoktur:

- JS sadece bridge'teki metodları görür (örneğin `process(text)`).
- Bridge içerden kütüphanenin public API'sini (`SmartEngine`, vb.) kullanır.
- Kütüphanenin kaç dosyadan oluştuğu, arkadaki import zinciri vb. JS tarafını ilgilendirmez.

Kodu projeye dahil etme seçenekleri

1. Kaynak kodu kopyalamak

- Tüm .kt dosyalarını `android/app/src/main/java/com/example/smartai/...` altına kopyalarsın.
- Paket adları ve klasör yapısı korunur.

2. Ayır Android library module olarak eklemek

- Android Studio ile projeyi `:smartai` modülü olarak içeri aktarırsın.
- `settings.gradle`:

```
include ':app', ':smartai'
```
- `android/app/build.gradle`:

```
dependencies {  
    implementation project(':smartai')  
}
```

3. Publish edilmiş Gradle dependency olarak kullanmak

- Eğer kütüphane Maven/JitPack üzerinde ise direkt:

```
dependencies {  
    implementation "com.github.user:smartai:1.0.0"  
}
```

Her üç durumda da bridge mantığı aynıdır.

Bridge örneği (çok dosyalı kütüphane)

```
import com.example.smartai.SmartEngine
import com.example.smartai.SmartConfig

class SmartBridgeModule(reactContext: ReactApplicationContext) :
    ReactContextBaseJavaModule(reactContext) {

    override fun getName(): String = "SmartBridgeModule"

    @ReactMethod
    fun process(text: String, promise: Promise) {
        try {
            val engine = SmartEngine(SmartConfig.default())
            val result = engine.process(text)
            promise.resolve(result)
        } catch (e: Exception) {
            promise.reject("SMART_ERROR", e)
        }
    }
}
```

- Kütüphanenin içinde Tokenizer, Normalizer, Logger vb. kaç sınıf olduğu JS için önemsizdir.
- JS sadece `SmartBridgeModule.process(text)` metodunu görür.

Kütüphanenin ekstra dependency'leri varsa

Örneğin kütüphane şunları kullanıyorsa:

```
implementation "com.squareup.okhttp3:okhttp:4.12.0"
implementation "com.google.code.gson:gson:2.10.1"
```

- Eğer ayrı module ise, bu bağımlılıklar o module'ün `build.gradle`'inde tanımlıdır.
- Eğer kaynak kodu doğrudan `app` modülüne kopyaladıysan, bu dependency'leri `android/app/build.gradle` içine eklemen gerekir.

JS API Tasarımını Sade Tutmak

Kütüphane çok karmaşık olabilir; ancak JS'e sadece ihtiyacın olan küçük bir API yüzeyi açmak yeterlidir:

```
type SmartAI = {  
  init(config: Config): Promise<void>;  
  analyze(text: string): Promise<SmartResult>;  
  reset(): Promise<void>;  
};
```

Native tarafta arka planda çok sayıda sınıf (`SmartConfig`, `Tokenizer`, `Cache` vb.) kullanılabilir, ancak:

- JS katmanı bunları görmez, sadece bridge fonksiyonlarını kullanır.
- Kütüphaneyi ileride değiştiren bile JS API sabit kalırsa, React Native kodun daha az kırılır.

Conclusion

Bu dokümanda, React tabanlı uygulamaların farklı sunum (rendering) modellerini ve bir React Native geliştiricisinin temas ettiği yerel (native) katmanı bütüncül bir bakış açısıyla ele alınmıştır. Sunucu taraflı üretim, artımlı statik yenileme, tek sayfa uygulaması yaklaşımı ve tamamen statik sayfa üretimi gibi yaklaşımların her biri, aynı bileşen mantığını farklı performans, ilk yükleme süresi ve içerik güncelliği gereksinimlerine göre konumlandıran stratejiler olarak özetlenmiştir.

React Native bağlamında ise, Android ve iOS taraflarının kavramsal temelleri (Application, Activity, Fragment, View, ViewGroup, UIViewController, UIView, Safe Area gibi yapılar), yerleşim ve yoğunluk kavramları (dp, ekran piksel yoğunluğu), ikon ve görsel yönetimi, manifest ve Info.plist yapılandırmaları ile Gradle ve CocoaPods tabanlı derleme sistemlerinin rolü sistematik biçimde ele alınmıştır. Buna ek olarak, JavaScript katmanı ile yerel katman arasındaki köprü mimarisi, Native Module ve View Manager kavramları; hem sıfırdan yazılan basit modüller hem de mevcut kütüphanelerin köprülenmesi üzerinden somut akışlarla örneklendirilmiştir. Bu sayede geliştiricinin yalnızca JavaScript tarafında üretken olmasının ötesine geçerek, gerektiğinde yerel kodu tasarlayabilen veya en azından doğru biçimde entegre edebilen bir zihinsel modele sahip olması amaçlanmıştır.

Sonuç olarak, burada sunulan çerçeve iki düzeyde pratik bir yol haritası sunmaktadır: Web tarafında, uygulamanın gereksinimlerine uygun sunum stratejisinin seçilebilmesi; mobil tarafta ise, React Native tabanlı bir uygulamanın Android ve iOS ekosistemleriyle dengeli ve bakımı görece kolay bir biçimde bütünleştirilebilmesi. Bu iki boyut birlikte ele alındığında, geliştirici hem son kullanıcının deneyimini hem de teknik mimarinin uzun vadeli sürdürülebilirliğini daha güvenle yönetebilecek bir konuma gelmektedir.